

SeuGado — Leo: o bot do Telegram

Entregar o plano no celular do produtor e registrar o que ele fez

27/09/2026

Sua parte em uma frase: o SeuGado mora no bolso do produtor. Toda segunda o plano chega pelo Telegram; com um toque, ele diz se fez, se não fez ou se fez diferente; e, quando o sistema pede, ele manda a altura do capim medida com régua. Quando ele não faz o que foi recomendado, o bot pede um plano novo e manda de volta.

Você entrega: `delivery/` (texto, canal Telegram, envio, respostas, conversa), `api/rotas_telegram.py` (webhook) e `scripts/telegram_polling.py` (modo de desenvolvimento).

Prazo: PR aberto até domingo à noite.

1. O SeuGado em um minuto

O gado é dividido em **lotes**, e o pasto em **piquetes**. O capim tem uma altura certa para o lote entrar e uma altura em que ele precisa sair. O sistema estima as alturas por satélite e clima, e o otimizador (João) monta o **plano da semana**. Você entrega esse plano e traz de volta a realidade: o que de fato aconteceu no campo. **A resposta “fiz diferente” é o dado mais valioso do sistema:** é onde o produtor discordou da máquina (ADR-007).

Princípio de produto (ADR-004): **confirmar tem que custar um toque**. Nada de digitar frase, nada de formulário. Toda interação é por botão, exceto a altura, que é um número.

Palavras que você vai usar

Termo	O que é
Plano (<code>PlanoManejo</code>)	O resultado do otimizador para 7 dias: movimentações, alertas e pedidos de medição
Movimentação	“Mover o lote X do piquete A para o B no dia D”. Tem um <code>id</code> único
Fiz	O produtor fez como recomendado → evento <code>manejo_confirmado</code>
Não fiz	Não fez → evento <code>manejo_recusado</code> → recalcular
Fiz diferente	Levou para outro piquete ou em outro dia → evento <code>manejo_divergente</code> → recalcular
Movimentação avulsa	O produtor mudou um lote sem recomendação → <code>manejo_confirmado</code> com id novo → recalcular
Confirmação por omissão	Se o dia passou e ele não respondeu, o sistema assume que fez (ADR-004), com confiança menor
Altura medida	O produtor mede o capim com régua e manda o número → evento <code>altura_medida</code>
Evento	Registro imutável de algo que aconteceu. Nunca se edita o passado

2. Stack e onde fica o seu código

Python 3.12, FastAPI e `httpx` para falar com a API do Telegram (sem biblioteca de bot, ADR-022). Sem dependência nova.

Arquivos que são seus (só você edita):

- `src/seugado/delivery/mensagem.py`: textos (funções puras)
- `src/seugado/delivery/canais/base.py`: interface Canal
- `src/seugado/delivery/canais/telegram.py`: implementação Telegram
- `src/seugado/delivery/envio.py`: `enviar_plano`
- `src/seugado/delivery/confirmacao.py`: registro das respostas + confirmação por omissão
- `src/seugado/delivery/bot.py`: a conversa (quem clicou em quê)
- `src/seugado/api/rotas_telegram.py`: webhook (já existe, vazio)
- `scripts/telegram_polling.py`
- `tests/delivery/*`

Você usa, mas não edita: `contratos.py` (`PlanoManejo`, `Movimentacao...`), `persistencia/eventos.py` (`registrar_evento`), `persistencia/projecao_db.py` (`reconstruir_projecao`), `persistencia/planos.py` (`carregar_plano_atual`, `do João`), `jobs/ciclo.py` (`executar_ciclo`, `do Leandro`), `api/deps.py`.

A interface `Canal` existe porque o WhatsApp vem depois (ADR-006): trocar de canal deve ser só escrever um novo adaptador, sem tocar no resto.

3. Preparar o ambiente

1. Instale Git e **uv** (docs.astral.sh/uv). Clone e crie a branch `feat/leo-telegram`. `uv sync --group dev`.

2. **Crie dois bots** no Telegram com o **@BotFather** (`/newbot`):

- `SeuGado` (produção; o webhook vai apontar para o Render);
- `SeuGado Dev` (o seu, para testar local com polling).

Não dá para usar webhook e polling no mesmo bot ao mesmo tempo. Guarde os dois tokens.

3. No BotFather, `/setcommands` nos dois bots: `plano - Ver o plano da semana · mover - Registrar que mudei um lote de piquete · ajuda - Como usar`.

4. `.env` (a partir do `.env.example`): `DATABASE_URL` (o Kauê manda em privado), `TELEGRAM_BOT_TOKEN` e `TELEGRAM_BOT_USERNAME` (do bot **Dev**, localmente) e `TELEGRAM_WEBHOOK_SECRET` (qualquer texto aleatório longo). Mande os dados do bot de **produção** ao Leandro, em privado, para o deploy.

5. Rode local com polling: `uv run python scripts/telegram_polling.py`.

Antes do PR: `uv run ruff check .`, `uv run mypy` e `uv run pytest` sem erro.

4. O que chega até você: o `PlanoManejo`

Exemplo completo em `tests/fixtures/plano_exemplo.json`. Carregue com `plano_de_dict(json.load(...))` e desenvolva tudo em cima dele. Campos que você usa:

```
plano.id, plano.fazenda_id, plano.data_inicio, plano.horizonte_dias
plano.movimentacoes: tuple[Movimentacao, ...]
    mov.id, mov.data, mov.lote_id, mov.lote_nome,
    mov.piquete_origem_id, mov.piquete_origem_nome,
    mov.piquete_destino_id, mov.piquete_destino_nome,
    mov.altura_destino_cm, mov.altura_entrada_alvo_cm, mov.dias_previstos,
    mov.motivo (frase pronta), mov.confianca ("alta"/"media"/"baixa"), mov.motivo_confianca
plano.alertas: tuple[Alerta, ...] # alerta.texto é frase pronta
plano.pedidos_validacao: tuple[PedidoValidacao, ...] # piquete_id, piquete_nome, motivo
```

5. A regra de gravação do projeto

Toda resposta do produtor vira **evento**, sempre com `ocorrido_em = datetime.now(UTC)` (a data real do fato vai **dentro** do payload), seguido de `reconstruir_projecao` e `commit`:

```
registrar_evento(conn, fazenda_id, TipoEvento.MANEJO_CONFIRMADO, OrigemEvento.PRODUTOR,
                 ocorrido_em=datetime.now(UTC), payload={...},
                 ator=f"telegram:{chat_id}", chave_idempotencia=f"resposta:{mov.id}")
reconstruir_projecao(conn, fazenda_id)
conn.commit()
```

Payloads (todos os campos obrigatórios; UUIDs e datas como texto ISO):

```
# Fiz → manejo_confirmado (entidade_id = id da movimentação)
{"entidade_id": mov.id, "lote_id": mov.lote_id, "piquete_destino_id":
mov.piquete_destino_id,
 "data_execucao": "2026-09-28"}
# Não fiz → manejo_recusado
{"entidade_id": mov.id, "motivo": None}
# Fiz diferente → manejo_divergente
{"entidade_id": mov.id, "lote_id": mov.lote_id, "piquete_real_id": "<uuid>",
 "data_execucao": "2026-09-29", "observacao": None}
# Movimentação avulsa → manejo_confirmado com entidade_id NOVO (uuid4), sem chave de
idempotência
{"entidade_id": "<uuid4>", "lote_id": "<uuid>", "piquete_destino_id": "<uuid>",
 "data_execucao": "2026-09-29"}
# Altura medida pelo bot → altura_medida
{"entidade_id": "<uuid4>", "piquete_id": "<uuid>", "data": "2026-09-29",
 "altura_cm": 35.0, "meio": "bot"}
```

`chave_idempotencia = f"resposta:{mov.id}"` em Fiz, Não fiz, Fiz diferente e na omissão: garanta **uma resposta por movimentação**, mesmo que o produtor toque duas vezes.

Tabelas que você lê ou escreve:

Tabela	Uso
fazenda	id, nome, timezone, telegram_chat_id, codigo_vinculo_telegram. Você faz UPDATE fazenda SET telegram_chat_id no /start

Tabela	Uso
estado_lote (derivada)	lote_id, nome, piquete_atual_id (lista de lotes no /mover; conferência da omissão)
estado_piquete (derivada)	piquete_id, nome, ativo, lote_atual_id (lista de piquetes)
evento	só leitura, para saber se uma movimentação já foi respondida: <code>SELECT tipo FROM evento WHERE fazenda_id = %s AND entidade_id = %s AND tipo IN ('manejo_confirmado', 'manejo_recusado', 'manejo_divergente')</code>
telegram_conversa	chat_id (PK), estado, dados (jsonb), atualizado_em. Guarda em que passo da conversa o produtor está

6. Contratos da sua parte (outros chamam)

```
# delivery/envio.py (o ciclo do Leandro chama)
def enviar_plano(conn, plano: PlanoManejo, atualizado: bool = False) -> bool
    # False se a fazenda não tem Telegram vinculado; True se enviou

# delivery/confirmacao.py (o ciclo do Leandro chama ANTES de gerar o plano)
def confirmar_por_omissao(conn, fazenda_id: UUID, hoje: date) -> int
    # quantas movimentações foram confirmadas por omissão; NÃO faz commit
```

E você chama o ciclo do Leandro para recalcular:

```
from seugado.jobs.ciclo import executar_ciclo
executar_ciclo(conn, fazenda_id, ingerir_satelite=False, atualizado=True)
# gera um plano novo, salva e manda com o cabeçalho "Plano atualizado"
```

7. Tarefas (na ordem)

T1: Interface e canal Telegram (canais/base.py, canais/telegram.py)

Por quê: isola o Telegram do resto. O WhatsApp entra depois só como outro adaptador.

Como:

```
@dataclass(frozen=True, slots=True)
class Botao:
    texto: str
    dados: str # callback_data: no máximo 64 bytes

class Canal(Protocol):
    def enviar_texto(self, chat_id: int, texto: str,
                    botoes: Sequence[Sequence[Botao]] = ()) -> None: ...
    def responder_clique(self, id_clique: str, texto: str | None = None) -> None: ...
```

CanalTelegram(token):

- enviar_texto → POST `https://api.telegram.org/bot<token>/sendMessage`, COM `{"chat_id", "text", "parse_mode": "HTML", "reply_markup": {"inline_keyboard": [{"text", "callback_data"}]}}` (sem

- `reply_markup` quando não há botões). Use `httpx.post(..., timeout=15)` e `raise_for_status()`.
- `responder_clique` → `answerCallbackQuery` com `callback_query_id` (e `text` opcional).
- `canal_padrao()` → `CanalTelegram` lê `TELEGRAM_BOT_TOKEN` do ambiente.
- Nome de piquete ou lote dentro de texto HTML passa por `html.escape`.
- Rejeite `Botao.dados` com mais de 64 bytes (`ValueError`).

Critérios de aceite:

- [] Testes com `httpx.post` falso conferem URL, corpo e teclado.
- [] Nenhum teste chama o Telegram de verdade.

Fora do escopo: fotos, áudio, localização, grupos.

T2: Textos (mensagem.py, puro)

Por quê: é o que o produtor lê. Regras do 07 §6: **sempre o motivo junto da ordem**, números em **cm** (nunca kg, NDVI ou termos técnicos) e **um plano por mensagem**.

Como:

```
def texto_plano(plano: PlanoManejo, fazenda_nome: str, atualizado: bool = False) -> str
def botoes_plano(plano: PlanoManejo) -> list[list[Botao]]
def texto_movimentacao(mov: Movimentacao) -> str
def botoes_movimentacao(mov: Movimentacao) -> list[list[Botao]]
```

Formato de `texto_plano` para `plano_exemplo.json` (é a referência; os seus testes devem conferir estas linhas):

```
<b>SeuGado – Fazenda Exemplo</b>
Plano da semana: seg 28/09 a dom 04/10

<b>Segunda, 28/09</b>
▶ <b>Mover Recria: Piquete 2 → Piquete 1</b>
0 Piquete 2 chegaria a 11,6 cm antes do próximo dia de manejo (qui 01/10), abaixo da saída de 15
cm. 0 Piquete 1 está em 31,5 cm (ponto de entrada: 30 cm).
Previsão: 3 dias no piquete · Confiança: média
<i>(a altura de entrada do Marandu no pastejo rotacionado vem de fonte com confiança média)</i>

▶ <b>Mover Vacas com bezerro: Piquete 5 → Piquete 6</b>
...

<b>Quinta, 01/10</b>
▶ <b>Mover Recria: Piquete 1 → Piquete 3</b>
...

<b>Avisos</b>
• 0 Piquete 8 (Mombaça) ainda não recebe recomendação: ...
• 0 Piquete 7 (pastejo contínuo) está em 38 cm, ...
• Nenhum piquete estará pronto para o lote Vacas com bezerro na quinta (01/10). ...

<b>Medições pedidas</b>
• Piquete 4: última imagem de satélite sem nuvem há 18 dias. Meça a altura com uma régua.

Toque abaixo para dizer o que você fez ou mandar uma medição.
```

- Com `atualizado=True`, a primeira linha vira `<b>Plano atualizado – {fazenda}</b>`.

- Plano sem movimentações: "Nenhuma movimentação necessária nesta semana." no lugar dos dias.
- Seções sem itens não aparecem.
- Limite do Telegram: 4.096 caracteres. Se passar, corte os avisos e termine com "... veja o plano completo no site."
- Dia por extenso: Segunda ... Domingo; abreviado: seg ter qua qui sex sáb dom.
- Confiança: alta / média / baixa.

botoes_plano:

- uma linha por movimentação: texto "{n}. {ddd dd/mm} · {lote} → {destino}", dados "m: {mov.id}";
- uma linha por pedido: texto "✎ Informar altura do {piquete}", dados "a:{piquete_id}".

botoes_movimentacao: uma linha com  Fiz ("f:{id}"),  Não fiz ("n:{id}") e  Fiz diferente ("d:{id}").

Critérios de aceite:

- [] texto_plano(plano_exemplo, "Fazenda Exemplo") reproduz as linhas acima.
- [] Nenhum texto contém "NDVI", "kg" ou "MS".
- [] Todos os dados de botão com ≤ 64 bytes.

Fora do escopo: emojis por tipo de alerta além dos acima; tradução.

T3: Envio (envio.py)

Por quê: é a porta que o ciclo semanal usa.

Como: enviar_plano(conn, plano, atualizado=False):

1. SELECT nome, telegram_chat_id FROM fazenda WHERE id = %s. Se telegram_chat_id for nulo, devolva False (sem erro: a fazenda ainda não vinculou).
2. canal_padrao().enviar_texto(chat_id, texto_plano(...), botoes_plano(plano)) e devolva True.

Aceite um canal: Canal | None = None opcional como último parâmetro, para os testes.

Critérios de aceite:

- [] Sem chat vinculado → False, sem chamada HTTP.

T4: Respostas e omissão (confirmacao.py)

Por quê: é o que fecha o ciclo (F-013). Sem saber o que o produtor fez, o próximo plano parte de uma fazenda que não existe.

Como:

```
def resposta_existente(conn, fazenda_id: UUID, mov_id: UUID) -> str | None # tipo do
evento ou None
def registrar_fiz(conn, fazenda_id, mov: Movimentacao, hoje: date, ator: str) -> None
def registrar_ao_fiz(conn, fazenda_id, mov: Movimentacao, ator: str) -> None
def registrar_diferente(conn, fazenda_id, mov: Movimentacao, piquete_real_id: UUID,
data_execucao: date, ator: str) -> None
```

```

def registrar_avulsa(conn, fazenda_id, lote_id: UUID, piquete_id: UUID, data_execucao:
date, ator: str) -> None
def registrar_altura(conn, fazenda_id, piquete_id: UUID, altura_cm: float, hoje: date,
ator: str) -> None
def confirmar_por_omissao(conn, fazenda_id: UUID, hoje: date) -> int

```

1. registrar_fiz: data_execucao = min(mov.data, hoje). Se ele tocou antes do dia, fez hoje; se tocou depois, vale o dia do plano.
2. Todas as funções de registro chamam `reconstruir_projecao`, mas **não** fazem commit: o `bot.py` faz.
3. registrar_diferente e registrar_avulsa: o piquete precisa existir, estar ativo e ser diferente do atual do lote; data_execucao não pode ser futura.
4. registrar_altura: $0 < altura_cm \leq 400$.
5. confirmar_por_omissao(conn, fazenda_id, hoje):
 - pegue o plano atual (`carregar_plano_atual`);
 - para cada movimentação com `mov.data < hoje`, sem resposta, e cujo lote ainda está no `piquete_origem_id` e cujo destino está vazio (`estado_lote`, `estado_piquete`), registre `manejo_confirmado` com `OrigemEvento.SISTEMA`, `ator="omissao"`, `data_execucao = mov.data` e a mesma chave `resposta:{mov.id}`;
 - devolva quantas foram confirmadas. O estado projetado usa `origem = sistema` para baixar a confiança do piquete.

Critérios de aceite:

- [] Tocar “Fiz” duas vezes gera um único evento.
- [] A omissão não confirma movimentação cujo lote já foi movido para outro lugar.
- [] A omissão ignora movimentações de hoje em diante.

T5: A conversa (bot.py)

Por quê: é onde os toques viram eventos, sem o produtor digitar nada além de um número.

Como: `processar_update(conn, update: dict, canal: Canal, hoje: date) -> None`. O `update` é o JSON que o Telegram manda.

Identificação:

- `chat_id` vem de `update["message"]["chat"]["id"]` ou de `update["callback_query"]["message"]["chat"]["id"]`.
- Fazenda: `SELECT id, nome FROM fazenda WHERE telegram_chat_id = %s`.
- Sem fazenda, só `/start <codigo>` funciona; qualquer outra coisa responde “Para conectar sua fazenda, abra o link da página Configurações do SeuGado.”

Mensagens de texto:

Texto	Ação
<code>/start <codigo></code>	<code>UPDATE fazenda SET telegram_chat_id = chat_id WHERE codigo_vinculo_telegram = codigo</code> → “Pronto! A {fazenda} está conectada. Toda segunda você recebe o plano da semana aqui.” Código inválido → mensagem de erro amigável

Texto	Ação
/plano	manda <code>texto_plano</code> + <code>botoes_plano</code> do plano atual (ou “ <i>Ainda não há plano.</i> ”)
/mover	inicia a movimentação avulsa (abaixo)
/ajuda	explica os 3 comandos e os botões
número (35, 35,5, 35 cm) com conversa no estado <code>altura</code>	<code>registrar_altura</code> → “ <i>Anotado: {piquete} com 35 cm hoje.</i> ”
qualquer outro	“ <i>Não entendi. Use /plano, /mover ou toque nos botões do plano.</i> ”

Cliques (sempre chame `responder_clique` primeiro):

Dados	Ação
<code>m:<id></code>	acha a movimentação no plano atual e manda <code>texto_movimentacao</code> + <code>botoes_movimentacao</code> . Não achou → “ <i>Esta movimentação é de um plano antigo. Use /plano.</i> ” Já respondida → “ <i>Você já respondeu esta movimentação.</i> ”
<code>f:<id></code>	<code>registrar_fiz</code> → commit → “ <i>✅ Anotado: {lote} no {destino}.</i> ”
<code>n:<id></code>	<code>registrar_nao_fiz</code> → commit → “ <i>Entendido. Vou refazer o plano da semana...</i> ” → recalcular
<code>d:<id></code>	pergunta “ <i>Para qual piquete o {lote} foi?</i> ”, com os piquetes ativos (menos a origem) como opções → depois “ <i>Quando?</i> ” (Hoje, Ontem, Anteontem) → <code>registrar_diferente</code> → commit → recalcular
<code>a:<piquete_id></code>	<code>estado_altura</code> → “ <i>Digite a altura do {piquete} em centímetros (só o número).</i> ”
<code>o:<n></code>	a opção número <code>n</code> do passo atual da conversa (veja abaixo)

Conversa com vários passos (`telegram_conversa`, um registro por chat, sobrescrito a cada passo com `INSERT ... ON CONFLICT (chat_id) DO UPDATE`):

estado	dados	Próximo passo
<code>divergente_piquete</code>	<code>{"mov_id", "opcoes": [piquete_ids]}</code>	<code>o:<n></code> → <code>divergente_data</code>
<code>divergente_data</code>	<code>{"mov_id", "piquete_id"}</code>	<code>o:0 / o:1 / o:2</code> = hoje / ontem / anteontem → registrar
<code>mover_lote</code>	<code>{"opcoes": [lote_ids]}</code>	<code>o:<n></code> → <code>mover_piquete</code>
<code>mover_piquete</code>	<code>{"lote_id", "opcoes": [piquete_ids]}</code>	<code>o:<n></code> → <code>mover_data</code>
<code>mover_data</code>	<code>{"lote_id", "piquete_id"}</code>	<code>o:0/1/2</code> → <code>registrar_avulsa</code> → commit → recalcular
<code>altura</code>	<code>{"piquete_id"}</code>	texto numérico → <code>registrar_altura</code> → commit

Ao terminar um fluxo, apague o estado. Opções como botões: uma por linha, texto = nome, dados = `o:<índice>`.

Recalcular: mande “*Recalculando o plano...*” e chame `executar_ciclo(conn, fazenda_id, ingerir_satelite=False, atualizado=True)`. Ele já salva e envia o plano novo. Se falhar, responda

“Não consegui recalcular agora. O plano novo chega na próxima segunda.” e registre o erro no log.

Critérios de aceite:

- [] Com um Canal falso e banco de teste (pulando se SEUGADO_TEST_DATABASE_URL não existir): /start vincula; f: gera manejo_confirmado; d: → o:1 → o:0 gera manejo_divergente e chama o recálculo (monkeypatch).
- [] Cliques de outro chat nunca mexem na fazenda errada.
- [] Nenhuma exceção sobe sem resposta ao produtor.

Fora do escopo: linguagem natural ou IA para interpretar texto (fica como ideia registrada); lembrete diário; foto de validação (F-017); vários usuários por fazenda.

T6: Webhook e polling (rotas_telegram.py, scripts/telegram_polling.py)

Por quê: em produção, o Telegram chama a nossa API (webhook). No seu computador, você busca as mensagens (polling).

Como:

1. POST /telegram/webhook:

- confira o cabeçalho X-Telegram-Bot-API-Secret-Token contra TELEGRAM_WEBHOOK_SECRET (diferente → 403);
- agende processar_update em BackgroundTasks, com **conexão própria** (abra e feche dentro da tarefa; não use a Conexao da rota);
- responda {"ok": true} na hora.

hoje = data atual em America/Fortaleza.

2. scripts/telegram_polling.py:

- chame deleteWebhook;
- em laço: getUpdates com offset e timeout=30; para cada update, abra uma conexão, chame processar_update e feche.

3. Registrar o webhook do bot de produção (depois do deploy do Leandro), uma vez, pelo navegador: https://api.telegram.org/bot<TOKEN>/setWebhook?url=https://<api>.onrender.com/telegram/webhook&secret_token=<SEGredo>. Confira com getWebhookInfo.

Critérios de aceite:

- [] Sem o cabeçalho certo → 403.
- [] O webhook responde em < 1 s, mesmo quando o recálculo demora.
- [] O polling local funciona com o bot Dev.

T7: Testes e PR

`tests/delivery/test_mensagem.py` (sem banco, com a fixture), `test_telegram.py` (HTTP falso), `test_confirmacao.py` e `test_bot.py` (banco de teste, pulando sem a variável). Depois, `ruff`, `mypy`, `pytest` e o PR `feat/leo-telegram` → `main`, com prints do bot.

8. Roteiro da demo (terça)

1. No site, Configurações → link do Telegram → o celular abre o bot → “Pronto!”
2. “Gerar plano agora” → o plano chega no celular.
3. Toque na movimentação de segunda → “Fiz diferente” → escolha outro piquete → “Hoje” → em segundos chega o **plano atualizado**.
4. Toque em “ Informar altura do Piquete 4” → digite 28 → “Anotado”.

9. Usando IA no seu fluxo

Dê à IA este documento, `src/seugado/contratos.py` e `tests/fixtures/plano_exemplo.json`, e peça uma tarefa por vez:

“Leia docs/equipe/LEO.md (seções 4, 5, 6 e a tarefa T2) e `contratos.py`. Implemente `src/seugado/delivery/mensagem.py` com funções puras que reproduzam o formato do exemplo. Não altere outros arquivos nem adicione dependências.”

Se a IA quiser usar `python-telegram-bot`, IA para interpretar texto, ou mexer em arquivo de outra pessoa, a resposta é não: fale com o Kauê.